
Спецификация

Компонент Eco.StdIO.C89

Статус: Черновик

Дата: Ноябрь 8, 2021

Версия: 1.0

Авторы	Компания
Владимир Башев	ПИРФ

Содержание

1. Обзор	4
1.1. Введение	4
1.2. Примечание	4
1.3. Ссылки	4
2. Компонент Eco.StdIO.C89	5
2.1. IEcoStdIOC89 описание на ECO IDL	6
2.1.1. Функция remove	8
2.1.2. Функция rename	8
2.1.3. Функция tmpfile	8
2.1.4. Функция tmpnam	8
2.1.5. Функция fclose	8
2.1.6. Функция fflush	8
2.1.7. Функция fopen	8
2.1.8. Функция freopen	9
2.1.9. Функция setbuf	9
2.1.10. Функция setvbuf	9
2.1.11. Функция fprintf	9
2.1.12. Функция fscanf	9
2.1.13. Функция printf	9
2.1.14. Функция scanf	9
2.1.15. Функция sprintf	10
2.1.16. Функция sscanf	10
2.1.17. Функция vfprintf	10
2.1.18. Функция vprintf	10
2.1.19. Функция vsprintf	10
2.1.20. Функция fgetc	10
2.1.21. Функция fgets	10
2.1.22. Функция fputc	11
2.1.23. Функция fputs	11
2.1.24. Функция getc	11
2.1.25. Функция getchar	11
2.1.26. Функция gets	11

2.1.27.	Функция <code>putc</code>	11
2.1.28.	Функция <code>putchar</code>	12
2.1.29.	Функция <code>puts</code>	12
2.1.30.	Функция <code>ungetc</code>	12
2.1.31.	Функция <code>fread</code>	12
2.1.32.	Функция <code>fwrite</code>	12
2.1.33.	Функция <code>fgetpos</code>	12
2.1.34.	Функция <code>fseek</code>	12
2.1.35.	Функция <code>fsetpos</code>	13
2.1.36.	Функция <code>ftell</code>	13
2.1.37.	Функция <code>rewind</code>	13
2.1.38.	Функция <code>clearerr</code>	13
2.1.39.	Функция <code>feof</code>	13
2.1.40.	Функция <code>ferror</code>	13
2.1.41.	Функция <code>perror</code>	13
Приложение А: Обучающие программы		14

1. Обзор

Данный документ описывает требования к реализации компонента Eco.StdIO.C89 (Компонент стандартная библиотека ввода/вывода).

1.1. Введение

Описание.

1.2. Примечание

- Ключевые слова в документе

1.3. Ссылки

Данный параграф содержит ссылки на информацию, помогающую понять данный документ:

[] – наименование ссылки

Доступен по: <http://адрес>

2. Компонент Eco.StdIO.C89

Компонент, имеет следующие описание:

[illegible]

```

int                vprintf                ([in] const char* format,
                                           [in] va_list arg);

int                vsprintf               ([in] char* s,
                                           [in] const char* format,
                                           [in] va_list arg);

int                fgetc                  ([in] FILE* stream);

char*              fgets                  ([in] char* s,
                                           [in] int c,
                                           [in] FILE* stream);

int                fputc                  ([in] int c,
                                           [in] FILE* stream);

int                fputs                   ([in] const char* s,
                                           [in] FILE* stream);

int                getc                   ([in] FILE* stream);

int                getchar                 ();

char*              gets                   ([in] char* s);

int                putc                   ([in] int c,
                                           [in] FILE* stream);

int                putchar                ([in] int c);

int                puts                   ([in] const char* s);

int                ungetc                 ([in] int c,
                                           [in] tFILE* stream);

size_t             fread                  ([in] void* ptr,
                                           [in] size_t size,
                                           [in] size_t nmemb,
                                           [in] FILE* stream);

size_t             fwrite                 ([in] const void* ptr,
                                           [in] size_t size,
                                           [in] size_t nmemb,
                                           [in] FILE* stream);

int                fgetpos                ([in] FILE* stream,
                                           [in] fpos_t* pos);

int                fseek                  ([in] FILE* stream,
                                           [in] long int offset,
                                           [in] int whence);

int                fsetpos                ([in] FILE* stream,
                                           [in] const fpos_t* pos);

long int           ftell                  ([in] FILE* stream);

void               rewind                 ([in] FILE* stream);

void               clearerr               ([in] FILE* stream);

int                feof                   ([in] FILE* stream);

int                ferror                 ([in] FILE* stream);

void               perror                 ([in] const char* s);

}

```

2.1.1. Функция remove

Библиотечная функция C удаляет заданное имя файла, чтобы он больше не был доступен.

2.1.2. Функция rename

Библиотечная функция C приводит к тому, что имя файла, на которое ссылается `old_filename`, изменяется на `new_filename`.

2.1.3. Функция tmpfile

Функция библиотеки C создает временный файл в режиме бинарного обновления (`wb+`). Созданный временный файл автоматически удаляется при закрытии потока (`fclose`) или при завершении работы программы.

2.1.4. Функция tmpnam

Библиотечная функция C генерирует и возвращает допустимое временное имя файла, которое не существует. Если `str` имеет значение `null`, то он просто возвращает имя файла `tmp`.

2.1.5. Функция fclose

Библиотечная функция C закрывает поток. Все буферы очищаются.

2.1.6. Функция fflush

Библиотечная функция C очищает выходной буфер потока.

2.1.7. Функция fopen

Библиотечная функция `C` открывает указанное имя файла по имени файла в заданном режиме.

2.1.8. Функция `freopen`

Библиотечная функция `C` связывает новое имя файла с данным открытым потоком и в то же время закрывает старый файл в потоке..

2.1.9. Функция `setbuf`

Библиотечная функция `C` определяет, как должен быть буферизован поток. Эта функция должна быть вызвана после того, как файл, связанный с потоком, уже открыт, но до того, как будет выполнена какая-либо операция ввода или вывода.

2.1.10. Функция `setvbuf`

Библиотечная функция `C` определяет, как поток должен быть буферизован.

2.1.11. Функция `fprintf`

Библиотечная функция `C` отправляет отформатированный вывод в поток.

2.1.12. Функция `fscanf`

Библиотечная функция `C` считывает отформатированные входные данные из потока.

2.1.13. Функция `printf`

Библиотечная функция `C` отправляет отформатированный вывод в `stdout`.

2.1.14. Функция `scanf`

Библиотечная функция `C` считывает отформатированный ввод из `stdin`.

2.1.15. Функция sprintf

Библиотечная функция C отправляет отформатированный вывод в строку, на которую указывает str..

2.1.16. Функция sscanf

Библиотечная функция C считывает отформатированные входные данные из строки.

2.1.17. Функция vfprintf

Библиотечная функция C отправляет отформатированный вывод в поток, используя переданный ему список аргументов..

2.1.18. Функция vprintf

Библиотечная функция C отправляет отформатированный вывод в stdout, используя переданный ей список аргументов..

2.1.19. Функция vsprintf

Библиотечная функция C отправляет отформатированный вывод в строку, используя переданный ей список аргументов.

2.1.20. Функция fgetc

Библиотечная функция C получает следующий символ (беззнаковый char) из указанного потока и перемещает индикатор позиции для потока.

2.1.21. Функция fgets

Библиотечная функция C считывает строку из указанного потока и сохраняет ее в строке, на которую указывает str. Он останавливается при чтении (n-1) символов, чтении

символа новой строки или достижении конца файла, в зависимости от того, что наступит раньше.

2.1.22. Функция fputc

Библиотечная функция C записывает символ (беззнаковый char), указанный аргументом char, в указанный поток и перемещает индикатор положения для потока.

2.1.23. Функция fputs

Библиотечная функция C записывает строку в указанный поток до нулевого символа, но не включая его.

2.1.24. Функция getc

Библиотечная функция C получает следующий символ (беззнаковый char) из указанного потока и перемещает индикатор позиции для потока.

2.1.25. Функция getchar

Библиотечная функция C получает символ (беззнаковый char) из stdin. Это эквивалентно getc с stdin в качестве аргумента.

2.1.26. Функция gets

Библиотечная функция C считывает строку из stdin и сохраняет ее в строке, на которую указывает str. Он останавливается при чтении символа новой строки или при достижении конца файла, в зависимости от того, что наступит раньше.

2.1.27. Функция putc

Библиотечная функция C записывает символ (беззнаковый char), заданный аргументом char, в указанный поток и перемещает индикатор позиции для потока.

2.1.28. Функция putchar

Библиотечная функция C записывает символ (беззнаковый char), указанный аргументом char, в stdout.

2.1.29. Функция puts

Библиотечная функция C записывает строку в stdout до нулевого символа, но не включая его. К выходным данным добавляется символ новой строки.

2.1.30. Функция ungetc

Библиотечная функция C помещает символ char (беззнаковый char) в указанный поток, чтобы this был доступен для следующей операции чтения.

2.1.31. Функция fread

Библиотечная функция C считывает данные из заданного потока в массив, на который указывает ptr.

2.1.32. Функция fwrite

Библиотечная функция C записывает данные из массива, на который указывает ptr в данный поток.

2.1.33. Функция fgetpos

Библиотечная функция C получает текущую позицию потока в файле и записывает ее в pos.

2.1.34. Функция fseek

Библиотечная функция C устанавливает позицию файла потока на заданное смещение.

2.1.35. Функция `fsetpos`

Библиотечная функция C устанавливает файловую позицию данного потока в заданную позицию. Аргумент `pos` — это позиция, задаваемая функцией `fgetpos`.

2.1.36. Функция `ftell`

Библиотечная функция C возвращает текущую позицию файла данного потока.

2.1.37. Функция `rewind`

Библиотечная функция C устанавливает позицию файла в начало файла данного потока.

2.1.38. Функция `clearerr`

Библиотечная функция C очищает индикаторы конца файла и ошибки для данного потока.

2.1.39. Функция `feof`

Библиотечная функция C проверяет индикатор конца файла для данного потока.

2.1.40. Функция `ferror`

Библиотечная функция C проверяет индикатор ошибки для данного потока.

2.1.41. Функция `perror`

Библиотечная функция C выводит описательное сообщение об ошибке в `stderr`. Сначала печатается строка `str`, затем двоеточие, а затем пробел.

Приложение А: Обучающие программы

```

/*
 * <кодировка символов>
 *   Cyrillic (UTF-8 with signature) - Codepage 65001
 * </кодировка символов>
 *
 * <сводка>
 *   EcoStdIOc89
 * </сводка>
 *
 * <описание>
 *   Данный исходный файл является точкой входа
 * </описание>
 *
 * <автор>
 *   Copyright (c) 2018 Vladimir Bashev. All rights reserved.
 * </автор>
 *
 */

/* Eco OS */
#include "IEcoSystem1.h"
#include "IdEcoMemoryManager1.h"
#include "IdEcoInterfaceBus1.h"
#include "IdEcoStdLibC89.h"
#include "IdEcoStdIOc89.h"
#include "IdEcoStringC89.h"
#include "IdEcoTimeC89.h"
#include "IdEcoThreadManager1.h"

/* Прототипы */
bool_t test_remove(IEcoStdIOc89* pStdIO);
bool_t test_remove(IEcoStdIOc89* pStdIO);
bool_t test_rename(IEcoStdIOc89* pStdIO);
bool_t test_tmpfile(IEcoStdIOc89* pStdIO);
bool_t test_tmpnam(IEcoStdIOc89* pStdIO);
bool_t test_fclose(IEcoStdIOc89* pStdIO);
bool_t test_fflush(IEcoStdIOc89* pStdIO, IEcoStringC89* pIString, IEcoThread1* pICurrentThread);
bool_t test_fopen(IEcoStdIOc89* pStdIO);
bool_t test_freopen(IEcoStdIOc89* pStdIO);
bool_t test_setbuf(IEcoStdIOc89* pStdIO);
bool_t test_setvbuf(IEcoStdIOc89* pStdIO, IEcoStringC89* pIString, IEcoThread1* pICurrentThread);
bool_t test_fprintf(IEcoStdIOc89* pStdIO);
bool_t test_fscanf(IEcoStdIOc89* pStdIO);
bool_t test_printf(IEcoStdIOc89* pStdIO);
bool_t test_scanf(IEcoStdIOc89* pStdIO);
bool_t test_sprintf(IEcoStdIOc89* pStdIO);
bool_t test_sscanf(IEcoStdIOc89* pStdIO, IEcoStringC89* pIString);
bool_t test_vfprintf(IEcoStdIOc89* pStdIO);
bool_t test_vprintf(IEcoStdIOc89* pStdIO);
bool_t test_vsprintf(IEcoStdIOc89* pStdIO);
bool_t test_fgetc(IEcoStdIOc89* pStdIO);
bool_t test_fgets(IEcoStdIOc89* pStdIO);
bool_t test_fputc(IEcoStdIOc89* pStdIO);

```

```

bool_t test_fputs(IEcoStdIOc89* pIStdIO);
bool_t test_getc(IEcoStdIOc89* pIStdIO);
bool_t test_getchar(IEcoStdIOc89* pIStdIO);
bool_t test_gets(IEcoStdIOc89* pIStdIO);
bool_t test_putc(IEcoStdIOc89* pIStdIO);
bool_t test_putchar(IEcoStdIOc89* pIStdIO);
bool_t test_puts(IEcoStdIOc89* pIStdIO, IEcoStringC89* pIString);
bool_t test_ungetc(IEcoStdIOc89* pIStdIO);
bool_t test_fread(IEcoStdIOc89* pIStdIO);
bool_t test_fwrite(IEcoStdIOc89* pIStdIO);
bool_t test_fgetpos(IEcoStdIOc89* pIStdIO);
bool_t test_fseek(IEcoStdIOc89* pIStdIO);
bool_t test_fsetpos(IEcoStdIOc89* pIStdIO);
bool_t test_ftell(IEcoStdIOc89* pIStdIO);
bool_t test_rewind(IEcoStdIOc89* pIStdIO);
bool_t test_clearerr(IEcoStdIOc89* pIStdIO);
bool_t test_feof(IEcoStdIOc89* pIStdIO);
bool_t test_ferror(IEcoStdIOc89* pIStdIO);
bool_t test_perror(IEcoStdIOc89* pIStdIO);

#define M_PI 3.14159265358979323846
char buffer[80];

void WriteFFrmt(IEcoStdIOc89* pIStdIO, FILE *stream, char *format, ...) {
    va_list args;

    va_start(args, format);
    pIStdIO->pVTbl->vfprintf(pIStdIO, stream, format, args);
    va_end(args);
}

void WriteFrmt(IEcoStdIOc89* pIStdIO, char *format, ...) {
    va_list args;

    va_start(args, format);
    pIStdIO->pVTbl->vprintf(pIStdIO, format, args);
    va_end(args);
}

int vspfunc(IEcoStdIOc89* pIStdIO, char *format, ...) {
    va_list aptr;
    int ret;

    va_start(aptr, format);
    ret = pIStdIO->pVTbl->vsprintf(pIStdIO, buffer, format, aptr);
    va_end(aptr);

    return(ret);
}

/*
 *
 * <сводка>
 *   Функция EcoMain
 * </сводка>
 *

```

```

* <описание>
*   Функция EcoMain - точка входа
* </описание>
*
*/
int16_t EcoMain(IEcoUnknown* pIUnk) {
    int16_t result = -1;
    /* Указатель на системный интерфейс */
    IEcoSystem1* pISys = 0;
    /* Указатель на интерфейс работы с системной интерфейсной шиной */
    IEcoInterfaceBus1* pIBus = 0;
    /* Указатель на интерфейс работы с памятью */
    IEcoMemoryAllocator1* pIMem = 0;
    char_t* name = 0;
    char_t* copyName = 0;
    /* Указатель на интерфейс стандартной библиотеки */
    IEcoStdLibC89* pIStdLib = 0;
    /* Указатель на интерфейс стандартного ввода/вывода */
    IEcoStdIOc89* pIStdIO = 0;
    /* Указатель на интерфейс для работы со строкой */
    IEcoStringC89* pIStrng = 0;
    /* Указатель на интерфейс для работы со временем */
    IEcoTimeC89* pITime = 0;
    /* Указатель на интерфейс работы с потоками */
    IEcoThreadManager1* pIThreadMgr = 0;
    IEcoThread1* pICurrentThread = 0;

    /* Проверка и создание системного интерфейса */
    if (pISys == 0) {
        result = pIUnk->pVTbl->QueryInterface(pIUnk, &IID_IEcoSystem1, (void **)&pISys);
        if (result != 0 && pISys == 0) {
            /* Освобождение системного интерфейса в случае ошибки */
            goto Release;
        }
    }

    /* Получение интерфейса для работы с интерфейсной шиной */
    result = pISys->pVTbl->QueryInterface(pISys, &IID_IEcoInterfaceBus1, (void **)&pIBus);
    if (result != 0 || pIBus == 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
}

#ifdef ECO_LIB
    /* Регистрация статического компонента для работы со стандартным вводом/выводом */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &IID_IEcoStdIOc89,
    (IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000000000000053494F31);
    if (result != 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы со строкой */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &IID_IEcoStringC89,
    (IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000000000000073747231);
    if (result != 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы со временем */

```



```

    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoTimeC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000544D4531);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы со стандартной библиотекой */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoStdLibC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_00000000000000000000000053544431);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы с менеджером потоков */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoThreadManager1,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_B7D8E10E1D3540CCB1CA96F766AF1936);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
#endif
    /* Получение интерфейса управления памятью */
    result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoMemoryManager1, 0, &IID_IEcoMemoryAllocator1, (void**)
&pIMem);

    /* Проверка */
    if (result != 0 && pIMem == 0) {
        /* Освобождение системного интерфейса в случае ошибки */
        goto Release;
    }

    /* Выделение блока памяти */
    name = (char_t *)pIMem->pVTbl->Alloc(pIMem, 10);

    /* Заполнение блока памяти */
    pIMem->pVTbl->Fill(pIMem, name, 'a', 9);

    /* Получение интерфейса для работы со строкой */
    pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStringC89, 0, &IID_IEcoStringC89, (void**) &pIString);
    if (result != 0 && pIString == 0) {
        /* Освобождение интерфейсов в случае ошибки */
        goto Release;
    }

    /* Получение интерфейса для работы со временем */
    pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoTimeC89, 0, &IID_IEcoTimeC89, (void**) &pITime);
    if (result != 0 && pITime == 0) {
        /* Освобождение интерфейсов в случае ошибки */
        goto Release;
    }

    /* Получение интерфейса для работы со стандартной библиотекой */
    pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStdLibC89, 0, &IID_IEcoStdLibC89, (void**) &pIStdLib);
    if (result != 0 && pIStdLib == 0) {
        /* Освобождение интерфейсов в случае ошибки */
        goto Release;
    }

```

```

/* Получение интерфейса для работы с менеджером потоков */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoThreadManager1, 0, &IID_IEcoThreadManager1, (void**)
&pIThreadMgr);
if (result != 0 && pIThreadMgr == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Получение интерфейса текущего потока */
pICurrentThread = pIThreadMgr->pVTbl->get_CurrentThread(pIThreadMgr);

/* Получение тестируемого интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStdIOc89, 0, &IID_IEcoStdIOc89, (void**) &pIStdIO);
if (result != 0 && pIStdIO == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Тестирование методов интерфейса */
test_remove(pIStdIO);
test_rename(pIStdIO);
test_tmpfile(pIStdIO);
test_tmpnam(pIStdIO);
test_fclose(pIStdIO);
//test_fflush(pIStdIO, pIString, pICurrentThread);
//test_fopen(pIStdIO);
//test_freopen(pIStdIO);
//test_setbuf(pIStdIO);
//test_setvbuf(pIStdIO, pIString, pICurrentThread);
//test_fprintf(pIStdIO);
//test_fscanf(pIStdIO);
test_printf(pIStdIO);
//test_scanf(pIStdIO);
//test_sprintf(pIStdIO);
//test_sscanf(pIStdIO, pIString);
//test_vfprintf(pIStdIO);
//test_vprintf(pIStdIO);
//test_vsprintf(pIStdIO);
//test_fgetc(pIStdIO);
//test_fgets(pIStdIO);
test_fputc(pIStdIO);
test_fputs(pIStdIO);
//test_getc(pIStdIO);
//test_getchar(pIStdIO);
//test_gets(pIStdIO);
test_putc(pIStdIO);
test_putchar(pIStdIO);
test_puts(pIStdIO, pIString);
test_ungetc(pIStdIO);
test_fread(pIStdIO);
test_fwrite(pIStdIO);
test_fgetpos(pIStdIO);
test_fseek(pIStdIO);
test_fsetpos(pIStdIO);
test_ftell(pIStdIO);
test_rewind(pIStdIO);
test_clearerr(pIStdIO);

```

```
test_feof(pIStdIO);
test_ferror(pIStdIO);
test_perror(pIStdIO);
```

```
/* Освобождение блока памяти */
pIMem->pVTbl->Free(pIMem, name);
```

Release:

```
/* Освобождение интерфейса для работы с интерфейсной шиной */
if (pIBus != 0) {
    pIBus->pVTbl->Release(pIBus);
}

/* Освобождение интерфейса работы с памятью */
if (pIMem != 0) {
    pIMem->pVTbl->Release(pIMem);
}

/* Освобождение интерфейса работы со временем */
if (pITime != 0) {
    pITime->pVTbl->Release(pITime);
}

/* Освобождение интерфейса работы со строкой */
if (pIString != 0) {
    pIString->pVTbl->Release(pIString);
}

/* Освобождение тестируемого интерфейса */
if (pIStdIO != 0) {
    pIStdIO->pVTbl->Release(pIStdIO);
}

/* Освобождение интерфейса работы со стандартной библиотекой */
if (pIStdLib != 0) {
    pIStdLib->pVTbl->Release(pIStdLib);
}

/* Освобождение интерфейса для работы с менеджером потоков */
if (pIThreadMgr != 0) {
    pIThreadMgr->pVTbl->Release(pIThreadMgr);
}

/* Освобождение системного интерфейса */
if (pISys != 0) {
    pISys->pVTbl->Release(pISys);
}

return result;
}
```

```
bool_t test_remove(IEcoStdIO89* pIStdIO) {
    int ret;
    FILE *fp;
    char filename[] = "file.txt";
```

```

fp = pIStdIO->pVTbl->fopen(pIStdIO, filename, "w");

pIStdIO->pVTbl->fprintf(pIStdIO, fp, "%s", "This is peerf.me");
pIStdIO->pVTbl->fclose(pIStdIO, fp);

ret = pIStdIO->pVTbl->remove(pIStdIO, filename);

if(ret == 0) {
    pIStdIO->pVTbl->printf(pIStdIO, "File deleted successfully");
} else {
    pIStdIO->pVTbl->printf(pIStdIO, "Error: unable to delete the file");
}

return 1;
}

bool_t test_rename(IEcoStdIO89* pIStdIO) {
    int ret;
    char oldname[] = "file.txt";
    char newname[] = "newfile.txt";

    ret = pIStdIO->pVTbl->rename(pIStdIO, oldname, newname);

    if(ret == 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "File renamed successfully");
    } else {
        pIStdIO->pVTbl->printf(pIStdIO, "Error: unable to rename the file");
    }
    return 1;
}

bool_t test_tmpfile(IEcoStdIO89* pIStdIO) {
    FILE *fp;

    fp = pIStdIO->pVTbl->tmpfile(pIStdIO);
    pIStdIO->pVTbl->printf(pIStdIO, "Temporary file created\n");

    /* you can use tmp file here */

    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_tmpnam(IEcoStdIO89* pIStdIO) {
    char buffer[L_tmpnam];
    char *ptr;

    pIStdIO->pVTbl->tmpnam(pIStdIO, buffer);
    pIStdIO->pVTbl->printf(pIStdIO, "Temporary name 1: %s\n", buffer);

    ptr = pIStdIO->pVTbl->tmpnam(pIStdIO, NULL);
    pIStdIO->pVTbl->printf(pIStdIO, "Temporary name 2: %s\n", ptr);
    return 1;
}

bool_t test_fclose(IEcoStdIO89* pIStdIO) {
    FILE *fp;

```

```

fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w");

pIStdIO->pVTbl->fprintf(pIStdIO, fp, "%s", "This is peerf.me");
pIStdIO->pVTbl->fclose(pIStdIO, fp);
return 1;
}

bool_t test_fflush(IEcoStdIOc89* pIStdIO, IEcoStringC89* pIString, IEcoThread1* pICurrentThread) {

    char buff[1024];

    pIString->pVTbl->memset(pIString, buff, '\\0', sizeof( buff ));

    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "Going to set full buffering on\\n");
    pIStdIO->pVTbl->setvbuf(pIStdIO, stdout, buff, _IOFBF, 1024);

    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "This is peerf.me\\n");
    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "This output will go into buff\\n");
    pIStdIO->pVTbl->fflush(pIStdIO, stdout );

    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "and this will appear when programm\\n");
    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "will come after sleeping 5 seconds\\n");

    pICurrentThread->pVTbl->Sleep(pICurrentThread, 5000);

    return 1;
}

bool_t test_fopen(IEcoStdIOc89* pIStdIO) {
    FILE * fp;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w+");
    pIStdIO->pVTbl->fprintf(pIStdIO, fp, "%s %s %s %d", "We", "are", "in", 2012);

    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    return 1;
}

bool_t test_freopen(IEcoStdIOc89* pIStdIO) {
    FILE *fp;

    pIStdIO->pVTbl->printf(pIStdIO, "This text is redirected to stdout\\n");

    fp = pIStdIO->pVTbl->freopen(pIStdIO, "file.txt", "w+", stdout);

    pIStdIO->pVTbl->printf(pIStdIO, "This text is redirected to file.txt\\n");

    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    return 1;
}

bool_t test_setbuf(IEcoStdIOc89* pIStdIO) {
    char buf[BUFSIZ];

    pIStdIO->pVTbl->setbuf(pIStdIO, stdout, buf);
    pIStdIO->pVTbl->puts(pIStdIO, "This is peerfme");
}

```

```

    pIStdIO->pVTbl->fflush(pIStdIO, stdout);
    return 1;
}

bool_t test_setvbuf(IEcoStdIO89* pIStdIO, IEcoStringC89* pIString, IEcoThread1* pICurrentThread) {
    char buff[1024];

    pIString->pVTbl->memset(pIString, buff, '\0', sizeof( buff ));

    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "Going to set full buffering on\n");
    pIStdIO->pVTbl->setvbuf(pIStdIO, stdout, buff, _IOFBF, 1024);

    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "This is peerf.me\n");
    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "This output will go into buff\n");
    pIStdIO->pVTbl->fflush(pIStdIO, stdout );

    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "and this will appear when programm\n");
    pIStdIO->pVTbl->fprintf(pIStdIO, stdout, "will come after sleeping 5 seconds\n");

    pICurrentThread->pVTbl->Sleep(pICurrentThread, 5000);
    return 1;
}

bool_t test_fprintf(IEcoStdIO89* pIStdIO) {
    FILE * fp;

    fp = pIStdIO->pVTbl->fopen (pIStdIO, "file.txt", "w+");
    pIStdIO->pVTbl->fprintf(pIStdIO, fp, "%s %s %s %d", "We", "are", "in", 2012);

    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_fscanf(IEcoStdIO89* pIStdIO) {
    char str1[10], str2[10], str3[10];
    int year;
    FILE * fp;

    fp = pIStdIO->pVTbl->fopen (pIStdIO, "file.txt", "w+");
    pIStdIO->pVTbl->fputs(pIStdIO, "We are in 2012", fp);

    pIStdIO->pVTbl->rewind(pIStdIO, fp);
    pIStdIO->pVTbl->fscanf(pIStdIO, fp, "%s %s %s %d", str1, str2, str3, &year);

    pIStdIO->pVTbl->printf(pIStdIO, "Read String1 |%s|\n", str1 );
    pIStdIO->pVTbl->printf(pIStdIO, "Read String2 |%s|\n", str2 );
    pIStdIO->pVTbl->printf(pIStdIO, "Read String3 |%s|\n", str3 );
    pIStdIO->pVTbl->printf(pIStdIO, "Read Integer |%d|\n", year );

    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_printf(IEcoStdIO89* pIStdIO) {
    int ch;

    for( ch = 75 ; ch <= 100; ch++ ) {

```

```

        pIStdIO->pVTbl->printf(pIStdIO, "ASCII value = %d, Character = %c\n", ch , ch );
    }
    return 1;
}

bool_t test_scanf(IEcoStdIO89* pIStdIO) {
    char str1[20], str2[30];

    pIStdIO->pVTbl->printf(pIStdIO, "Enter name: ");
    pIStdIO->pVTbl->scanf(pIStdIO, "%19s", str1);

    pIStdIO->pVTbl->printf(pIStdIO, "Enter your website name: ");
    pIStdIO->pVTbl->scanf(pIStdIO, "%29s", str2);

    pIStdIO->pVTbl->printf(pIStdIO, "Entered Name: %s\n", str1);
    pIStdIO->pVTbl->printf(pIStdIO, "Entered Website:%s", str2);

    return 1;
}

bool_t test_sprintf(IEcoStdIO89* pIStdIO) {
    char str[80];

    pIStdIO->pVTbl->sprintf(pIStdIO, str, "Value of Pi = %f", M_PI);
    pIStdIO->pVTbl->puts(pIStdIO, str);
    return 1;
}

bool_t test_sscanf(IEcoStdIO89* pIStdIO, IEcoStringC89* pIString) {
    int day, year;
    char weekday[20], month[20], dtm[100];

    pIString->pVTbl->strcpy(pIString, dtm, "Saturday March 25 1989" );
    pIStdIO->pVTbl->sscanf(pIStdIO, dtm, "%s %s %d %d", weekday, month, &day, &year );

    pIStdIO->pVTbl->printf(pIStdIO, "%s %d, %d = %s\n", month, day, year, weekday );

    return 1;
}

bool_t test_vfprintf(IEcoStdIO89* pIStdIO) {
    FILE *fp;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt","w");

    WriteFFrmt(pIStdIO, fp, "This is just one argument %d \n", 10);

    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_vprintf(IEcoStdIO89* pIStdIO) {
    WriteFrmt(pIStdIO, "%d variable argument\n", 1);
    WriteFrmt(pIStdIO, "%d variable %s\n", 2, "arguments");
    return 1;
}

bool_t test_vsprintf(IEcoStdIO89* pIStdIO) {

```

```

int i = 5;
float f = 27.0;
char str[50] = "peerf.me";

vfprintf(pIStdIO, "%d %f %s", i, f, str);
pIStdIO->pVTbl->printf(pIStdIO, "%s\n", buffer);

return 1;
}

bool_t test_fgetc(IEcoStdIOC89* pIStdIO) {
    FILE *fp;
    int c;
    int n = 0;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    if(fp == NULL) {
        pIStdIO->pVTbl->perror(pIStdIO, "Error in opening file");
        return(-1);
    } do {
        c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break ;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
    } while(1);

    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_fgets(IEcoStdIOC89* pIStdIO) {
    FILE *fp;
    char str[60];

    /* opening file for reading */
    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt" , "r");
    if(fp == NULL) {
        pIStdIO->pVTbl->perror(pIStdIO, "Error opening file");
        return(-1);
    }
    if( pIStdIO->pVTbl->fgets (pIStdIO, str, 60, fp)!=NULL ) {
        /* writing content to stdout */
        pIStdIO->pVTbl->puts(pIStdIO, str);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_fputc(IEcoStdIOC89* pIStdIO) {
    FILE *fp;
    int c;
    int ch;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w+");
    for( ch = 33 ; ch <= 100; ch++ ) {
        pIStdIO->pVTbl->fputc(pIStdIO, ch, fp);
    }
}

```



```

pIStdIO->pVTbl->fclose(pIStdIO, fp);

fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
while(1) {
    c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
    if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
        break ;
    }
    pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
}
pIStdIO->pVTbl->fclose(pIStdIO, fp);
return 1;
}

bool_t test_fputs(IEcoStdIO89* pIStdIO) {
    FILE *fp;
    int c;
    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "wt");

    pIStdIO->pVTbl->fputs(pIStdIO, "This is c programming.", fp);
    pIStdIO->pVTbl->fputs(pIStdIO, "This is a system programming language.", fp);

    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    while(1) {
        c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break ;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_getc(IEcoStdIO89* pIStdIO) {
    char c;

    pIStdIO->pVTbl->printf(pIStdIO, "Enter character: ");
    c = pIStdIO->pVTbl->getc(pIStdIO, stdin);
    pIStdIO->pVTbl->printf(pIStdIO, "Character entered: ");
    pIStdIO->pVTbl->putc(pIStdIO, c, stdout);
    return 1;
}

bool_t test_getchar(IEcoStdIO89* pIStdIO) {
    char c;

    pIStdIO->pVTbl->printf(pIStdIO, "Enter character: ");
    c = pIStdIO->pVTbl->getchar(pIStdIO);

    pIStdIO->pVTbl->printf(pIStdIO, "Character entered: ");
    pIStdIO->pVTbl->putchar(pIStdIO, c);
    return 1;
}

bool_t test_gets(IEcoStdIO89* pIStdIO) {

```

```

char str[50];

pIStdIO->pVTbl->printf(pIStdIO, "Enter a string : ");
pIStdIO->pVTbl->gets(pIStdIO, str);

pIStdIO->pVTbl->printf(pIStdIO, "You entered: %s", str);
return 1;
}

bool_t test_putc(IEcoStdIOc89* pIStdIO) {
    FILE *fp;
    int ch;
    int c;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w");
    for( ch = 33 ; ch <= 100; ch++ ) {
        pIStdIO->pVTbl->putc(pIStdIO, ch, fp);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    while(1) {
        c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break ;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    return 1;
}

bool_t test_putchar(IEcoStdIOc89* pIStdIO) {
    char ch;

    for(ch = 'A' ; ch <= 'Z' ; ch++) {
        pIStdIO->pVTbl->putchar(pIStdIO, ch);
    }
    return 1;
}

bool_t test_puts(IEcoStdIOc89* pIStdIO, IEcoStringC89* pIString) {
    char str1[15];
    char str2[15];

    pIString->pVTbl->strcpy(pIString, str1, "peerf");
    pIString->pVTbl->strcpy(pIString, str2, "compileonline");

    pIStdIO->pVTbl->puts(pIStdIO, str1);
    pIStdIO->pVTbl->puts(pIStdIO, str2);
    return 1;
}

bool_t test_ungetc(IEcoStdIOc89* pIStdIO) {
    FILE *fp;
    int c;

```

```

char buffer [256];

fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
if( fp == NULL ) {
    pIStdIO->pVTbl->perror(pIStdIO, "Error in opening file");
    return(-1);
}
while(!pIStdIO->pVTbl->feof(pIStdIO, fp)) {
    c = getc (fp);
    /* replace ! with + */
    if( c == '!' ) {
        pIStdIO->pVTbl->ungetc (pIStdIO, '+', fp);
    } else {
        pIStdIO->pVTbl->ungetc(pIStdIO, c, fp);
    }
    pIStdIO->pVTbl->fgets(pIStdIO, buffer, 255, fp);
    pIStdIO->pVTbl->fputs(pIStdIO, buffer, stdout);
}
return 1;
}

bool_t test_fread(IEcoStdIOc89* pIStdIO) {
    FILE *fp;
    char c[] = "this is peerfme";
    char buffer[100];

    /* Open file for both reading and writing */
    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w+");

    /* Write data to the file */
    pIStdIO->pVTbl->fwrite(pIStdIO, c, strlen(c) + 1, 1, fp);

    /* Seek to the beginning of the file */
    pIStdIO->pVTbl->fseek(pIStdIO, fp, 0, SEEK_SET);

    /* Read and display data */
    pIStdIO->pVTbl->fread(pIStdIO, buffer, strlen(c)+1, 1, fp);
    pIStdIO->pVTbl->printf(pIStdIO, "%s\n", buffer);
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_fwrite(IEcoStdIOc89* pIStdIO) {
    FILE *fp;
    char str[] = "This is peerf.me";
    int c;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w");
    pIStdIO->pVTbl->fwrite(pIStdIO, str, 1, sizeof(str), fp);

    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    while(1) {
        c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break;
        }
    }
}

```

```

    }
    pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
}
pIStdIO->pVTbl->fclose(pIStdIO, fp);
return 1;
}

bool_t test_fgetpos(IEcoStdIOc89* pIStdIO) {
    FILE *fp;
    fpos_t position;
    int c;
    int n = 0;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w+");
    pIStdIO->pVTbl->fgetpos(pIStdIO, fp, &position);
    pIStdIO->pVTbl->fputs(pIStdIO, "Hello, World!", fp);

    pIStdIO->pVTbl->fsetpos(pIStdIO, fp, &position);
    pIStdIO->pVTbl->fputs(pIStdIO, "This is going to override previous content", fp);
    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");

    while(1) {
        c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break ;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
    }

    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_fseek(IEcoStdIOc89* pIStdIO) {
    FILE *fp;
    int c;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w+");
    pIStdIO->pVTbl->fputs(pIStdIO, "This is peerf.me", fp);

    pIStdIO->pVTbl->fseek(pIStdIO, fp, 7, SEEK_SET );
    pIStdIO->pVTbl->fputs(pIStdIO, " C Programming Language", fp);
    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    while(1) {
        c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

```

```

}

bool_t test_fsetpos(IEcoStdIO89* pIStdIO) {
    FILE *fp;
    fpos_t position;
    int c;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w+");
    pIStdIO->pVTbl->fgetpos(pIStdIO, fp, &position);
    pIStdIO->pVTbl->fputs(pIStdIO, "Hello, World!", fp);

    pIStdIO->pVTbl->fsetpos(pIStdIO, fp, &position);
    pIStdIO->pVTbl->fputs(pIStdIO, "This is going to override previous content", fp);
    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    while(1) {
        c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_ftell(IEcoStdIO89* pIStdIO) {
    FILE *fp;
    int len;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    if( fp == NULL ) {
        pIStdIO->pVTbl->perror (pIStdIO, "Error opening file");
        return(-1);
    }
    pIStdIO->pVTbl->fseek(pIStdIO, fp, 0, SEEK_END);

    len = pIStdIO->pVTbl->ftell(pIStdIO, fp);
    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    pIStdIO->pVTbl->printf(pIStdIO, "Total size of file.txt = %d bytes\n", len);

    return 1;
}

bool_t test_rewind(IEcoStdIO89* pIStdIO) {
    char str[] = "This is peerf.me";
    FILE *fp;
    int ch;

    /* First let's write some content in the file */
    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w");
    pIStdIO->pVTbl->fwrite(pIStdIO, str, 1, sizeof(str), fp);
    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    while(1) {

```

```

        ch = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break ;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", ch);
    }
    pIStdIO->pVTbl->rewind(pIStdIO, fp);
    pIStdIO->pVTbl->printf(pIStdIO, "\n");
    while(1) {
        ch = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break ;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", ch);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_clearerr(IEcoStdIOc89* pIStdIO) {
    FILE *fp;
    char c;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w");

    c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
    if( pIStdIO->pVTbl->ferror(pIStdIO, fp) ) {
        pIStdIO->pVTbl->printf(pIStdIO, "Error in reading from file : file.txt\n");
    }
    pIStdIO->pVTbl->clearerr(pIStdIO, fp);

    if( pIStdIO->pVTbl->ferror(pIStdIO, fp) ) {
        pIStdIO->pVTbl->printf(pIStdIO, "Error in reading from file : file.txt\n");
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);

    return 1;
}

bool_t test_feof(IEcoStdIOc89* pIStdIO) {
    FILE *fp;
    int c;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    if(fp == NULL) {
        pIStdIO->pVTbl->perror(pIStdIO, "Error in opening file");
        return(-1);
    }

    while(1) {
        c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
        if( pIStdIO->pVTbl->feof(pIStdIO, fp) ) {
            break ;
        }
        pIStdIO->pVTbl->printf(pIStdIO, "%c", c);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
}

```

```

    return 1;
}

bool_t test_ferror(IEcoStdIO89* pIStdIO) {
    FILE *fp;
    char c;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "w");

    c = pIStdIO->pVTbl->fgetc(pIStdIO, fp);
    if( pIStdIO->pVTbl->ferror(pIStdIO, fp) ) {
        pIStdIO->pVTbl->printf(pIStdIO, "Error in reading from file : file.txt\n");
    }
    pIStdIO->pVTbl->clearerr(pIStdIO, fp);

    if( pIStdIO->pVTbl->ferror(pIStdIO, fp) ) {
        pIStdIO->pVTbl->printf(pIStdIO, "Error in reading from file : file.txt\n");
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

bool_t test_perror(IEcoStdIO89* pIStdIO) {
    FILE *fp;

    /* first rename if there is any file */
    pIStdIO->pVTbl->rename(pIStdIO, "file.txt", "newfile.txt");

    /* now let's try to open same file */
    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    if( fp == NULL ) {
        pIStdIO->pVTbl->perror(pIStdIO, "Error: ");
        return(-1);
    }
    pIStdIO->pVTbl->fclose(pIStdIO, fp);
    return 1;
}

```